PROJECT SYNOPSIS

on

HIERARCHICAL REINFORCEMENT LEARNING AND EXPLAINABLE AI FOR MULTI-AGENT AUTONOMOUS DECISION SYSTEMS

Submitted to

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD

In partial fulfillment of the requirements for the award of the degree of

MASTER OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Srinivas Rao Tammireddy

23R21A0501

Under the Guidance of

Dr. S. Pratap Singh

Professor, Department of Computer Science and Engineering

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Marri Laxman Reddy Institute of Technology and Management

Dundigal, Hyderabad – 500 043, Telangana, India

Academic Year: 2024–2025

1. Title of the Project

Hierarchical Reinforcement Learning and Explainable AI for Multi-Agent Autonomous Decision Systems

2. Abstract

Autonomous multi-agent coordination in dynamic, partially observable environments presents profound challenges for conventional reinforcement learning methods, particularly regarding scalability, decision transparency, and cooperative efficiency. This project proposes an Explainable Multi-Agent Hierarchical Reinforcement Learning framework — implemented as the EMARL-SAR system — for cooperative search and rescue (SAR) operations in dynamic grid environments. The framework deploys three heterogeneous autonomous agents, namely two reconnaissance drones (A1 and A2) and one rescue rover (A3), within a 15×15 discrete grid representing a simulated disaster zone containing hidden victims, spreading fire hazards, debris obstacles, and charge stations.

Agents learn cooperative rescue strategies through a two-level hierarchical policy architecture. The high-level policy employs a tabular Q-table with epsilon-greedy selection and Semi-Markov Decision Process (SMDP) updates to assign strategic macro-goals, including quadrant exploration, charge station routing, victim retrieval targeting, and debris clearance targeting. The low-level policy employs a Deep Q-Network (DQN) with a 64-64 multi-layer perceptron architecture combined with a Breadth-First Search (BFS) pathfinder for reliable step-by-step primitive action selection.

A dedicated Explainability Engine is tightly integrated throughout the framework, generating real-time decision justifications at three levels: high-level strategic rationales incorporating peer coordination awareness, low-level spatial movement explanations with explicit hazard avoidance characterization, and gradient-based saliency attributions computed using PyTorch autograd that decompose each tactical decision into percentage contributions from target proximity, battery status, and hazard avoidance features. The complete system is deployed as a Python HTTP server with a live browser-based HTML dashboard for real-time simulation visualization and training monitoring.

Experimental evaluation demonstrates a full rescue success rate of 66–100% compared to 0–33% for a flat DQN baseline, with agent survival rates of 67–100% and substantially improved battery efficiency, confirming the practical effectiveness of the hierarchical approach.

3. Introduction

Search and rescue operations in disaster-stricken environments are among the most operationally demanding and time-critical applications of autonomous multi-agent systems. Natural disasters such as earthquakes, building collapses, and industrial fires create environments that are inherently dangerous for human first responders, dynamically evolving as secondary hazards spread, and partially observable due to structural obstructions and limited sensor ranges. Deploying autonomous agents — unmanned aerial vehicles for aerial reconnaissance and ground robots for physical debris clearance and victim extraction — offers a promising approach to augment human SAR capabilities while minimizing life risk to human responders.

Multi-Agent Reinforcement Learning (MARL) provides a principled framework for enabling agents to discover cooperative strategies through interaction with the environment. However, conventional flat MARL approaches face fundamental limitations in SAR contexts. The joint action space grows exponentially with agent count, making flat policy learning intractable for large teams. Learned neural network policies operate as opaque black boxes, providing no insight into the reasoning behind individual decisions, which is unacceptable in safety-critical domains where human coordinators must be able to supervise, understand, and override autonomous behavior.

Hierarchical Reinforcement Learning (HRL) addresses the scalability barrier through temporal decomposition, separating strategic macro-goal assignment from tactical primitive action execution. Explainable AI (XAI) addresses the transparency barrier by generating human-comprehensible justifications of agent decisions in real time. The proposed framework unifies HRL and XAI within a single architecture purpose-built for heterogeneous multi-agent SAR coordination. The three agents — two reconnaissance drones and one rescue rover — possess complementary capabilities: drones fly over debris to rapidly scan sectors and detect victims, while the rover systematically clears debris and physically rescues scanned victims. This role complementarity drives the cooperative value of the proposed heterogeneous multi-agent design.

The framework's live browser dashboard with gradient saliency bars, spatial justification logs, and SSE-streamed training metrics enables real-time operational transparency for human coordinators, directly addressing the trust and accountability requirements of autonomous system deployment in safety-critical contexts.

4. Problem Statement

Autonomous multi-agent coordination for search and rescue operations in dynamic grid environments faces a combination of unsolved challenges that conventional flat MARL approaches cannot address simultaneously. These challenges include:

- Exponential Joint Action Space: As the number of cooperating agents increases, the joint action space grows exponentially, making flat policy learning computationally intractable and severely degrading sample efficiency.
- Credit Assignment Complexity: In cooperative environments with sparse rescue rewards and long temporal horizons (up to 150 steps per episode), assigning credit for successful victim rescues backward through extended action sequences is extremely difficult without hierarchical decomposition.
- Black-Box Opacity of Learned Policies: Neural network-based MARL policies provide no interpretable account of why specific actions were selected, preventing effective human supervision, post-incident auditing, and regulatory compliance in safety-critical deployments.
- Dynamic Hazard Adaptation: Stochastically spreading fire hazards continuously change the traversability of the grid, rendering path plans obsolete and requiring continuous hazard-aware route adaptation that rule-based planners cannot provide.
- Heterogeneous Capability Exploitation: Effective SAR coordination requires explicitly leveraging the distinct capabilities of different agent types (drone aerial traversal over debris vs. rover debris clearance and victim rescue), which flat homogeneous MARL architectures cannot model.
- Battery Resource Management: Each agent operates with a finite battery resource (100 units) that depletes with every action. Effective coordination must balance rescue progress against proactive battery management, requiring strategic-level energy planning beyond the scope of primitive action policies.

The primary problem addressed in this project is therefore the design and implementation of a multi-agent coordination framework that simultaneously resolves all six challenges through hierarchical policy decomposition and embedded real-time explainability, enabling transparent, efficient, and trustworthy autonomous SAR coordination in dynamic grid environments.

5. Objectives

- To design and implement a two-level hierarchical reinforcement learning framework for cooperative multi-agent search and rescue in a dynamic grid environment.
- To integrate a real-time Explainability Engine that generates human-interpretable justifications and gradient-based saliency attributions for all agent decisions.
- To evaluate the framework performance through rescue success metrics and comparative analysis against non-hierarchical baselines.

6. Literature Review

S. No	Author	Title and Description	Technique Used	Limitations
1	K. Zhang, Z. Yang, and T. Basar (2021)	Multi-Agent Reinforcement Learning: A Selective Overview — Comprehensive survey of cooperative, competitive, and mixed-motive MARL theories, covering credit assignment, non-stationarity, and scalability challenges.	Cooperative Markov Games, Independent Q-Learning, Centralized Training with Decentralized Execution (CTDE)	Does not address explainability requirements. No hierarchical temporal abstraction. Scalability remains a bottleneck for large joint action spaces.
2	Y. Yu, Z. Zhai, W. Li, and J. Ma (2024)	Target-Oriented Multi-Agent Coordination with HRL — Proposes a hierarchical RL framework for cooperative navigation that assigns target coordinates to individual	Hierarchical Reinforcement Learning, High-Level Goal Assignment, Low-Level Navigation Policy	Assumes homogeneous agents without role-specific capabilities. Static environment without dynamic hazard propagation. No explainability

		agents at the macro-goal level.		mechanisms provided.
3	P. Feng et al. (2024)	Hierarchical Consensus-Based MARL for Multi-Robot Cooperation — Introduces graph-based communication for consensus formation at the strategic level to coordinate multi-robot task assignments.	Graph Attention Network, Consensus Protocol, Hierarchical MARL	Requires extensive inter-agent communication bandwidth. Consensus overhead limits real-time applicability. No decision explanation support.
4	R. S. Sutton, D. Precup, and S. Singh (1999)	Between MDPs and Semi-MDPs: The Options Framework — Foundational work establishing temporal abstraction for RL through options (initiation set, intra-option policy, termination condition) and SMDP Q-learning updates.	Semi-Markov Decision Process, Options Framework, Temporal Abstraction, Q-Learning	Single-agent framework without multi-agent extension. No explainability or heterogeneous agent support. Options require manual specification in early formulations.
5	V. Mnih et al. (2015)	Human-Level Control Through Deep Reinforcement Learning — Introduces the Deep Q-Network (DQN) with experience replay and	Deep Q-Network (DQN), Experience Replay Buffer, Target Network, Convolutional Neural Network	Designed for single-agent settings. No multi-agent coordination or hierarchical decomposition. Learned policies remain black-box without

		target network stabilization achieving human-level performance on complex sequential tasks.		interpretability mechanisms.
6	A. Adadi and M. Berrada (2018)	Peeking Inside the Black-Box: A Survey on XAI — Comprehensive survey categorizing XAI methods by scope, model dependency, and explanation type; identifies gradient-based attribution as computationally efficient for real-time scenarios.	Gradient-Based Saliency, SHAP, LIME, Feature Attribution, Post-Hoc Explanation Methods	Survey work without concrete RL implementation. Post-hoc methods not designed for real-time deployment. SHAP computation cost prohibitive for per-step explanation generation.

7. Proposed Methodology

The proposed methodology follows a systematic five-stage pipeline:

Data Collection → Environment Design → Hierarchical Policy Development → Explainability Integration → Training, Evaluation, and Deployment

Stage 1: Environment Design and Configuration

The SearchRescueEnv class implements a 15×15 discrete NumPy grid with five cell types: empty (0), debris/obstacle (1), fire (2), and charge station (3). Two charge stations are placed at fixed positions on opposite grid sides. Ten debris obstacles and two fire seed locations are randomly spawned at episode initialization. Three hidden victims are placed with random positions following a hidden → scanned → rescued lifecycle. Stochastic fire spread executes every 7 timesteps with an 18% probability per adjacent empty cell.

Stage 2: High-Level Policy Design (Q-Table with SMDP)

The HighLevelAgent implements tabular Q-learning with agent-specific state discretization. Drone states are (battery_low, unexplored, overlap) and rover states are (battery_low, scanned_victim_exists, debris_exists), each yielding 8 discrete state tuples. The drone macro-goal space has 5 actions (4 quadrant centers + nearest charge station) and the rover has 7 (4 quadrants + charge station + closest scanned victim + closest debris). Epsilon-greedy selection with decay schedule max(0.05, 0.15 × 0.95^(ep/10)) balances exploration and exploitation. SMDP Bellman updates accumulate rewards across entire option executions before performing Q-table updates.

Stage 3: Low-Level Policy Design (DQN + BFS)

The LowLevelAgent uses a 12-dimensional state vector: normalized sub-goal displacement (dx, dy), normalized battery level, and 9-element 3×3 local grid surroundings encoding cell types. The QNetwork architecture is Linear(12→64) → ReLU → Linear(64→64) → ReLU → Linear(64→5) with separate networks for drones and rover. BFS pathfinding is the primary action selection mechanism, applying role-specific traversal constraints (drones block fire; rover blocks fire and debris in primary pass). The DQN policy serves as fallback when BFS cannot find valid paths. Experience replay (5,000 transitions) and target network synchronization (every 5 episodes) stabilize learning.

Stage 4: Explainability Engine Integration

The ExplainabilityEngine generates three complementary explanation types in real time. High-level justifications combine strategic rationale from translate_action_to_goal(), current battery status, and peer coordination context. Low-level spatial justifications describe movement direction relative to sub-goal approach, characterizing actions as reducing/maintaining/increasing distance and identifying fire or debris in neighboring cells. Gradient saliency attributions use PyTorch autograd: the absolute gradient of the greedy action's Q-value with respect to input features is grouped into Target Proximity (features 0–1), Battery Status (feature 2), and Hazard Avoidance (features 3–11), normalized to 100%.

Stage 5: Training, Server Deployment, and Evaluation

The SMDP training loop in main.py trains for 50 episodes using synchronized high-level SMDP updates and per-step low-level DQN gradient updates. The HTTP server (server.py) provides three API endpoints: /api/reset, /api/simulate (one step with full explanations), and /api/train (SSE streaming of per-epoch metrics). The live dashboard renders the grid, agent positions, exploration fog-of-war, explanation logs, and gradient saliency bar charts. Performance is evaluated against a flat DQN baseline using rescue success rate, agent survival, completion steps, and battery efficiency.

8. System Architecture / Framework

The EMARL-SAR system architecture is organized into five integrated modules communicating through a well-defined data pipeline:

SearchRescueEnv (environment.py) → HighLevelAgent (high_level_agent.py) → LowLevelAgent (low_level_agent.py) → ExplainabilityEngine (explain.py) → SARServerHandler + Dashboard (server.py + live_demo.html)

Module Descriptions

1. Search and Rescue Environment: 15×15 NumPy grid with dynamic fire spread, hidden victim discovery lifecycle (hidden → scanned → rescued), debris obstacles, charge stations, cooperative reward structure (+40 for victim rescue, +12 debris clearance, +5 recharge, −25 fire collision, −20 battery depletion, −0.1 step penalty), and three agent roles (A1/A2 drones at grid corners, A3 rover at bottom center).
2. High-Level Agent: Tabular Q-table with 8-state discrete representation per agent type. Macro-goal space: 5 actions for drones (4 quadrant centers [3,3], [3,11], [11,3], [11,11] + charge station), 7 actions for rover (drone goals + closest scanned victim + closest debris). SMDP Q-updates with learning rate $\alpha = 0.15$, discount $\gamma = 0.9$.
3. Low-Level Agent: DQN (QNetwork: 12→64→64→5) with BFS-primary navigation. BFS applies role-specific traversal constraints. Experience replay buffer (5,000 transitions), mini-batch size 64, Adam optimizer (lr = 1e-3), discount $\gamma = 0.95$, target network sync every 5 episodes. Separate drone and rover network weights.
4. Explainability Engine: Three explanation modalities — strategic high-level textual justification, spatial low-level movement justification with hazard detection, and real-time PyTorch autograd gradient saliency grouped into Target Proximity / Battery Status / Hazard Avoidance percentage attributions.
5. Server and Dashboard: Python http.server with CORS support, REST endpoints (/api/reset, /api/simulate, /api/train), SSE streaming for live training metrics, and HTML5 dashboard with canvas grid rendering, explanation log panel, and saliency visualization bars.

9. Algorithms and Technologies Used

Core Algorithms

- Q-Learning with SMDP (Semi-Markov Decision Process): High-level macro-goal assignment using tabular Q-table. Update rule: $Q(s,o) \leftarrow Q(s,o) + \alpha[R_{acc} + \gamma \cdot \max_{o'} Q(s',o') - Q(s,o)]$, where R_{acc} is the accumulated reward over the entire option execution and k is the number of primitive steps taken.
- Deep Q-Network (DQN): Low-level primitive action selection. Loss: $L(\theta) = E[(r + \gamma \cdot \max_{a'} Q_{\theta'}(s',a') - Q_{\theta}(s,a))^2]$. Stabilized via experience replay and periodic target network synchronization.
- Breadth-First Search (BFS): Primary navigation mechanism for shortest obstacle-aware path finding. Agent-specific traversal constraints: drones block fire cells; rover blocks fire and debris in primary BFS, fire only in secondary BFS for clearing sequence planning.
- Gradient Saliency Attribution: Real-time feature importance using PyTorch autograd. $Saliency_j = |\partial Q_{\theta}(s, a^*)/\partial s_j|$. Grouped into three semantic categories and normalized to percentage attributions summing to 100%.
- Epsilon-Greedy Exploration with Exponential Decay: Shared schedule $\max(0.05, 0.15 \times 0.95^{(ep/10)})$ for both policy levels, transitioning from exploration-dominant to exploitation-dominant behavior.

Technologies Used

- Python 3.10+: Primary implementation language for all environment, policy, explainability, training, and server modules.
- PyTorch 2.x with Autograd: DQN neural network training and real-time gradient saliency computation via automatic differentiation.
- NumPy 1.24+: 15×15 integer grid representation, state vector construction, and Q-table numerical operations.
- Python http.server + socketserver: Built-in HTTP server for REST API endpoints and SSE streaming without external dependencies.
- HTML5 / CSS3 / JavaScript (Fetch API + EventSource): Browser-based live dashboard with canvas grid rendering, real-time SSE metric streaming, and explanation log display.
- pickle: Standard library serialization for Q-table weight persistence across training sessions.
- collections.deque: Fixed-size experience replay buffer for efficient DQN transition sampling.

10. Dataset Description

The EMARL-SAR framework is trained and evaluated entirely on synthetic data generated by the SearchRescueEnv simulation engine. Each training episode generates a unique grid configuration through randomized placement of debris, fire, and victims, ensuring diverse training experience without reliance on fixed datasets. The dataset characteristics are as follows:

Parameter	Value / Description
Environment Type	Simulated 15×15 discrete grid (225 cells total)
Episode Count (Training)	50 training episodes per run (SMDP hierarchical loop)
Max Steps per Episode	150 timesteps (early termination on all rescued or all dead)
Victim Count	3 per episode (randomized positions, 3-stage lifecycle)
Debris Obstacles	10 per episode (randomized in grid interior rows 2–12)
Fire Seeds	2 per episode with 18% stochastic spread every 7 steps
State Vector Dimension	12 elements: (dx, dy, battery, fov[0..8])
Replay Buffer Capacity	5,000 transitions (shared across drone and rover agents)
Action Space	5 primitive actions: North, South, East, West, Interact
High-Level State Space	8 discrete states per agent type (3-element binary tuple)
Training / Evaluation Split	50 training episodes + independent evaluation over 20 test episodes

11. Experimental Setup

Hardware and Software Environment

- Hardware: Intel Core i5 / AMD Ryzen 5 or equivalent (8 GB RAM minimum; NVIDIA CUDA GPU optional for accelerated DQN training)
- Software: Python 3.10+, PyTorch 2.x (CPU/CUDA), NumPy 1.24+, Google Chrome 110+ for dashboard
- Server: Python built-in http.server on localhost:8000 with CORS enabled

Hyperparameter Configuration

Parameter	Value	Component
Q-Table Learning Rate (α)	0.15	High-Level Q-Table
High-Level Discount (γ_H)	0.90	High-Level Q-Table
DQN Learning Rate (Adam)	1e-3	Low-Level DQN
Low-Level Discount (γ_L)	0.95	Low-Level DQN
Replay Buffer Size	5,000	Low-Level DQN
Mini-Batch Size	64	Low-Level DQN
Target Network Sync Interval	Every 5 eps	Low-Level DQN
Initial / Minimum Epsilon	0.15 / 0.05	Both Levels
DQN Architecture	12→64→64→5	Low-Level DQN

Performance Metrics

- Rescue Success Rate: Percentage of victims (0/3, 1/3, 2/3, 3/3) successfully rescued per episode.
- Average Steps to Completion: Number of timesteps to rescue all 3 victims (episodes achieving complete rescue only).
- Agent Survival Rate: Percentage of agents (out of 3) remaining active at episode end.
- Battery Efficiency: Average remaining battery across all active agents at episode end.
- Explanation Generation Success Rate: Percentage of agent decision steps producing valid explanation outputs.

12. Expected Results

Based on experimental runs, the proposed EMARL-SAR framework is expected to achieve the following performance outcomes:

Metric	EMARL-SAR (Proposed)	Flat DQN Baseline
Full Rescue Success Rate (3/3)	66 – 100%	0 – 33%
Partial Success (≥1 victim rescued)	100%	33 – 66%

Avg. Steps to Full Completion	85 – 120 steps	130 – 150 steps
Agent Survival Rate	67 – 100%	33 – 67%
Battery Efficiency (avg remaining)	30 – 55 units	5 – 20 units
Explanation Generation Success	100%	N/A

Additional Expected Outcomes

- Effective sector distribution: Drones will learn to assign themselves to distinct quadrants, minimizing redundant exploration and maximizing victim detection coverage.
- Proactive battery management: High-level policy will learn to trigger charge station macro-goals before battery depletion, sustaining operational capability across full episode duration.
- Coherent explanation quality: High-level justifications will accurately reflect strategic rationale; gradient saliency attributions will show Target Proximity dominance when far from sub-goals and Hazard Avoidance elevation near fire zones.
- Training convergence: Rescue success rates will improve progressively from 0–33% in early episodes to 66–100% after episode 35, with DQN training loss decreasing from ~4.0 to ~1.0 over 50 episodes.

13. Conclusion

This project presents a complete Explainable Multi-Agent Hierarchical Reinforcement Learning framework for autonomous cooperative search and rescue coordination in dynamic, partially observable grid environments. The proposed EMARL-SAR system uniquely integrates three capabilities that no existing approach in the literature satisfies simultaneously: hierarchical temporal decomposition for scalable multi-agent coordination, embedded real-time explainability at strategic and tactical levels, and heterogeneous agent role design that exploits complementary drone and rover capabilities.

The two-level hierarchical architecture — Q-table high-level macro-goal assignment with SMDP updates and DQN+BFS low-level primitive action selection — effectively reduces the SAR coordination problem into manageable sub-problems at each level of abstraction, producing measurably superior rescue success rates, agent survival rates, and battery efficiency compared to flat DQN baselines. The Explainability Engine's three-modality output (strategic justifications, spatial movement explanations,

and gradient saliency percentage attributions) provides the transparency required for trusted autonomous system deployment in safety-critical domains.

The complete end-to-end implementation — from the dynamic 15×15 grid environment through the hierarchical policies and explainability engine to the HTTP server and live browser dashboard — provides a self-contained research platform and practical demonstration tool. The project contributes directly to the growing field of trustworthy autonomous multi-agent AI by establishing that hierarchical decision-making and real-time explainability are complementary and mutually beneficial design principles for cooperative autonomous systems.

14. Timeline / Work Plan

Phase	Activity	Duration
Phase 1	Literature Survey — Review of MARL, HRL, XAI, and SAR-related research works; identification of research gap	2 Weeks
Phase 2	Environment Design — Implementation of SearchRescueEnv (15×15 grid, fire spread, victim lifecycle, reward structure)	2 Weeks
Phase 3	High-Level Policy Development — Q-table design, state discretization, SMDP update rule, macro-goal space definition	2 Weeks
Phase 4	Low-Level Policy Development — DQN architecture, BFS pathfinder with role-specific constraints, experience replay, target network	3 Weeks
Phase 5	Explainability Engine — High-level justification, low-level spatial explanation, PyTorch autograd gradient saliency implementation	2 Weeks
Phase 6	Server and Dashboard — HTTP server API endpoints (/reset, /simulate, /train), SSE streaming, HTML5 live dashboard	2 Weeks
Phase 7	Training, Testing, and Evaluation — SMDP training loop, hyperparameter	3 Weeks

	tuning, baseline comparison, metric analysis	
Phase 8	Documentation and Report Writing — Project report, synopsis, and presentation preparation	2 Weeks

15. References

[1] K. Zhang, Z. Yang, and T. Basar, "Multi-agent reinforcement learning: A selective overview of theories and algorithms," in Handbook of Reinforcement Learning and Control, Springer, 2021, pp. 321–384.

[2] Y. Yu, Z. Zhai, W. Li, and J. Ma, "Target-Oriented Multi-Agent Coordination with Hierarchical Reinforcement Learning," Applied Sciences, vol. 14, no. 16, p. 7084, Aug. 2024, doi: 10.3390/app14167084.

[3] P. Feng et al., "Hierarchical Consensus-Based Multi-Agent Reinforcement Learning for Multi-Robot Cooperation Tasks," in Proc. IEEE/RSJ IROS, Oct. 2024, pp. 642–649, doi: 10.1109/IROS58592.2024.10802212.

[4] R. S. Sutton, D. Precup, and S. Singh, "Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning," Artificial Intelligence, vol. 112, no. 1–2, pp. 181–211, 1999.

[5] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017, pp. 4765–4774.

[6] V. Mnih et al., "Human-level control through deep reinforcement learning," Nature, vol. 518, no. 7540, pp. 529–533, Feb. 2015, doi: 10.1038/nature14236.

[7] T. Rashid, M. Samvelyan, C. S. de Witt, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning," in Proc. ICML, PMLR, 2018, pp. 4295–4304.

[8] A. Adadi and M. Berrada, "Peeking inside the black-box: A survey on explainable artificial intelligence (XAI)," IEEE Access, vol. 6, pp. 52138–52160, 2018.

[9] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[10] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in Advances in NeurIPS, vol. 30, 2017.

[11] J. Chen, J. Sun, and G. Wang, "From unmanned systems to autonomous intelligent systems," Engineering, vol. 12, pp. 16–19, 2022, doi: 10.1016/j.eng.2021.10.007.

[12] S. Manaseswaran et al., "Swarm Robotics in Search and Rescue Operations: Challenges, Strategies, and Future Directions," in Proc. 10th Int. Conf. Smart Structures and Systems (ICSSS), IEEE, 2025, pp. 1–6.